

Salute di Ferro

Guida Copywriter & Workflow GitHub

Documento operativo per gestire l'aggiornamento dei contenuti del sito **salutediferro.com** in collaborazione tra dev (Trilli) e copywriter, con deploy automatico in produzione.

Repo: github.com/Salutediferro/SITO

Stack: React 19 + Vite 8 + Vercel (auto-deploy)

Versione: 1.1 — 28 aprile 2026 (post-cleanup orfani)

Indice

- 1. Mappa del copy nel codice
- 2. Cosa può modificare il copywriter (e cosa no)
- 3. Setup GitHub per il copywriter
- 4. Workflow di aggiornamento (passo-passo)
- 5. Branch protection e deploy automatico Vercel
- 6. Casi d'uso concreti
- 7. Errori comuni da evitare
- 8. Checklist rapida (ogni modifica)
- Appendice A — Glossario
- Appendice B — Comandi utili

1. Mappa del copy nel codice

Il copy del sito SDF è organizzato in 3 livelli di accessibilità per il copywriter, a seconda del contesto del file. Più il file è in **verde**, più è sicuro modificarlo senza rischi di rompere il sito.

■ Livello SAFE — file constants/

File di soli dati (testo strutturato in oggetti JavaScript). Modifiche immediate, zero rischio di rompere logica. **L'80% del copy del sito è qui.**

File	Cosa contiene	Visibile in
<code>frontend/src/constants/faq.js</code>	9 FAQ con domanda (q) + risposta (a)	Sezione FAQ in homepage
<code>frontend/src/constants/panels.js</code>	Pannelli FERRO CORE + MODULES (FERRO_DONNA, ANDROGENO, CUORE, RENI, FEGATO, METABOLICO, TIROIDE, RECOVERY, DONNA): nome, prezzo, biomarker, descrizioni, highlights	Pagina /pannelli + componente PanelShowcase home
<code>frontend/src/constants/hashtags.js</code>	Tag descrittivi (#TESTOSTERONE, #CUORE, ecc.) + funzione computeTags	Risultato finale del quiz
<code>frontend/src/constants/quiz.js</code>	Domande del quiz, opzioni risposta, label, codici referral validi	Test di Ferro (/test)

■ Livello INTERMEDIO — components/

File di componenti React con testo "inline" dentro markup JSX. Modifiche possibili ma serve attenzione: il copy è circondato da tag `<p>`, `<h2>`, `<span>`. **Modificare SOLO il testo tra i tag**, mai i tag stessi.

File	Cosa contiene
<code>frontend/src/components/home/Hero.jsx</code>	Titolo H1 home, sottotitolo, label bottoni promo, card prezzi promo
<code>frontend/src/components/home/HowItWorks.jsx</code>	Step "come funziona" sulla homepage
<code>frontend/src/components/home/ClientProof.jsx</code>	Sezione social proof clienti (homepage)
<code>frontend/src/components/home/TrustBadges.jsx</code>	Badge fiducia (homepage)
<code>frontend/src/components/home/PanelShowcase.jsx</code>	Carosello pannelli (homepage). Legge dati da panels.js
<code>frontend/src/components/home/FAQ.jsx</code>	Wrapper sezione FAQ (homepage). Legge da faq.js
<code>frontend/src/components/layout/Navbar.jsx</code>	Etichette menu navigazione (HOME, PANNELLI, CHI SIAMO, WHATSAPP, MEMBERSHIP, TEST DI FERRO)

File	Cosa contiene
frontend/src/components/layout/Footer.jsx	Footer copy + tagline brand
frontend/src/components/quiz/IntroStep.jsx	Step iniziale quiz
frontend/src/components/quiz/SingleChoiceStep.jsx	Step quiz con singola scelta
frontend/src/components/quiz/MultiChoiceStep.jsx	Step quiz con scelte multiple
frontend/src/components/quiz/ContactsStep.jsx	Step quiz raccolta contatti
frontend/src/components/quiz/EndStep.jsx	Step finale quiz con esito personalizzato
frontend/src/components/quiz/PaymentStep.jsx	Step quiz con CTA pagamento
frontend/src/components/ui/PricePromo.jsx	Componente card prezzo promo riusabile
frontend/src/components/ui/Accordion.jsx	Componente FAQ accordion (markup, non testo FAQ)

■ Livello HARD — pages/

Pagine intere con testo lungo + layout complesso. **Suggerito che il copywriter modifichi solo se ha esperienza JSX**, altrimenti consegnare il nuovo testo in un documento separato e Trilli applica.

File	Cosa contiene	Note
frontend/src/pages/HomePage.jsx	Composizione homepage	Solo struttura, raramente copy
frontend/src/pages/ChiSiamoPage.jsx	Pagina /chi-siamo: hero, problemi, 3 pilastri, sezioni storia	Tanto copy inline
frontend/src/pages/MembershipPage.jsx	Pagina /membership: hero, sezione promo €197, vantaggi, servizi inclusi, downsell	Tutto copy hardcoded inline (no constants)
frontend/src/pages/PannelliPage.jsx	Pagina /pannelli: catalogo + filosofia evidence-based	Mix: alcuni dati da panels.js, altri inline
frontend/src/pages/TestPage.jsx	Wrapper pagina /test (apre il quiz)	Solo struttura, no copy
frontend/src/pages/PrivacyPage.jsx	■ Informativa privacy GDPR	NON modificare senza OK Giovanni (legale)

2. Cosa può modificare il copywriter (e cosa no)

■ Sì — copywriter può modificare

- ✓ Testo dentro virgolette singole o doppie nei file constants/
- ✓ Domande e risposte FAQ (faq.js)
- ✓ Descrizioni pannelli (panels.js)
- ✓ Descrizioni e vantaggi membership (membership.js)
- ✓ Domande e label quiz (quiz.js)
- ✓ Tagline e descrizioni componenti home (componenti components/home/*.jsx) — solo testo dentro tag
- ✓ Titoli pagine, sottotitoli, paragrafi (con cura nei file pages/)
- ✓ Etichette bottoni (es. "FAI IL TEST DI FERRO")
- ✓ Etichette navbar / footer

■ No — riservato a dev (Trilli)

- ✗ **Privacy / Termini / Cookie** (PrivacyPage.jsx) — solo con OK Giovanni (legale)
- ✗ Logica scoring quiz (scoring.js) — sono numeri, non testo
- ✗ URL Stripe Payment Links (payments.js)
- ✗ Codici referral validi (quiz.js → VALID_REFERRALS)
- ✗ Stili CSS, palette colori (styles/globals.css)
- ✗ Tutti i file dentro src/ ROOT (Cloudflare Worker, non frontend)
- ✗ package.json, vite.config.js, wrangler.toml
- ✗ Qualsiasi codice JavaScript con function, const di logica, import, export

Regola d'oro

Il copywriter modifica **SOLO testo tra virgolette** ('...' o "...") o **tra apertura e chiusura tag JSX** (<p>testo</p>). Mai toccare nulla che assomigli a function, const, =>, {...}, import, export.

3. Setup GitHub per il copywriter

3.1 Aggiungere il copywriter al repo (azione Trilli)

Una volta sola, dal browser su GitHub:

1. Vai su github.com/Salutediferro/SITO/settings/access
2. Click **Add people**
3. Inserisci username GitHub o email del copywriter
4. Ruolo: **Write** (non Admin — il copywriter non deve cambiare settings repo)
5. Conferma. Il copywriter riceverà invito email.

3.2 Setup macchina copywriter (azione copywriter)

Una volta sola, sulla propria macchina:

A. Installare Git (se non presente)

macOS: già installato. Verifica con:

```
git --version
```

Windows: scaricare da git-scm.com

B. Configurare identità Git

```
git config --global user.name "Nome Copywriter"  
git config --global user.email "email@copywriter.com"
```

C. Clonare il repo

```
cd ~/Desktop  
git clone https://github.com/Salutediferro/SITO.git  
cd SITO
```

D. Editor consigliato: VS Code

Scarica code.visualstudio.com. Aprire la cartella SITO con: File → Open Folder → SITO/. Tutti i file sono navigabili dalla sidebar.

Estensioni VS Code utili: **GitLens** (cronologia visiva), **ESLint** (avvisa errori sintassi).

4. Workflow di aggiornamento (passo-passo)

Ogni modifica copy segue questo flusso. Tempo medio per piccola modifica: **5-10 minuti** dal testo finale al sito live.

Step 1 — Allineare il repo locale (copywriter)

Prima di iniziare a lavorare, scaricare le ultime modifiche da GitHub:

```
cd ~/Desktop/SITO
git checkout main
git pull origin main
```

Step 2 — Creare un branch dedicato

Mai modificare direttamente main. Creare un branch con nome descrittivo (prefisso copy/ + breve descrizione):

```
git checkout -b copy/aggiorna-faq-prezzo-2026
# oppure
git checkout -b copy/nuova-tagline-membership
```

Step 3 — Modificare il file con VS Code

Aprire il file dalla sidebar VS Code, trovare il testo (Cmd/Ctrl + F per cercare), modificarlo, salvare (Cmd/Ctrl + S). Esempio modifica FAQ:

```
// PRIMA
{ q: 'Quanto costa?', a: 'Il Test di Ferro costa 27€...' }

// DOPO
{ q: 'Quanto costa?', a: 'Il Test di Ferro costa 29€...' }
```

Step 4 — Committare le modifiche

```
git add frontend/src/constants/faq.js
git commit -m "copy: aggiorna prezzo Test di Ferro a 29€ nelle FAQ"
```

Convenzione messaggio commit: copy: ... (sempre prefisso **copy**) + descrizione breve italiano del cambiamento.

Step 5 — Pubblicare il branch su GitHub

```
git push -u origin copy/aggiorna-faq-prezzo-2026
```

Step 6 — Aprire una Pull Request (PR)

Andare su github.com/Salutediferro/SITO, cliccare il banner giallo "Compare & pull request". Riempire titolo + descrizione e submit.

Step 7 — Vercel genera Preview Link automatico

Entro ~1 minuto, Vercel posta un commento sulla PR con un link <https://sito-xxx.vercel.app>. È il **preview live del branch**: il copywriter (e Trilli) possono vedere come apparirà la modifica prima del merge.

Step 8 — Review Trilli + merge

Trilli rivede la PR: clicca "Files changed" su GitHub, controlla il diff, poi se OK clicca "Approve" + "Merge pull request".

Step 9 — Vercel deploys in produzione (automatico)

Al merge in main, Vercel riavvia il build e pubblica su salutediferro.com. Tempo: ~2-3 minuti.

Step 10 — Pulizia (opzionale)

Tornare su main e cancellare il branch locale già mergiato:

```
git checkout main  
git pull origin main  
git branch -d copy/aggiorna-faq-prezzo-2026
```


5. Branch protection e deploy automatico Vercel

5.1 Branch protection su main (azione Trilli, una volta sola)

Settings GitHub per evitare push diretti su main:

- Settings → Branches → Add branch protection rule
- Branch name pattern: main
- ☒ Require a pull request before merging
- ☒ Require approvals: 1 (Trilli)
- ☒ Require status checks: Vercel Preview
- ☒ Do not allow bypassing the above settings
- Save

5.2 Vercel auto-deploy (già attivo)

Il progetto SDF su Vercel è già collegato al repo GitHub. Configurazione attuale:

Trigger	Cosa succede
Push su qualsiasi branch	Build + deploy preview su URL temporaneo
Merge in main	Build + deploy produzione su salutediferro.com
PR aperta	Vercel commenta la PR con link preview cliccabile

5.3 Tempi reali

Operazione	Tempo
Modifica file + commit + push locale	1-2 min
Apertura PR su GitHub	30 sec
Build Vercel preview	1-2 min
Review Trilli	variabile (5-30 min)
Merge + deploy produzione	2-3 min
Totale tipico end-to-end	10-40 min

6. Casi d'uso concreti

■ ATTENZIONE — File da NON modificare (orfani / legacy)

Esiste storia tecnica nel repo. Questi file sono **già stati rimossi** nella versione 1.0 di questo documento, ma se per errore vedi nomi simili in branch vecchi o cloni obsoleti del repo, **NON li modificare**: erano file orfani mai serviti al sito.

File rimosso	Motivo
<code>constants/membership.js</code>	Definiva 4 livelli MEMBERSHIP ma MembershipPage.jsx non lo importava — copy mai mostrato
<code>components/home/ChiSiamo.jsx</code>	Mai importato. Versione attiva in pages/ChiSiamoPage.jsx
<code>components/home/Features.jsx</code>	Mai importato
<code>components/home/FerroCore.jsx</code>	Mai importato. Spotlight FERRO_CORE è in PanelShowcase.jsx
<code>components/home/FerroDonna.jsx</code>	Mai importato
<code>components/home/ModulesGrid.jsx</code>	Mai importato
<code>components/home/SocialProof.jsx</code>	Mai importato
<code>components/home/Stats.jsx</code>	Mai importato
<code>components/home/Testimonials.jsx</code>	Mai importato
<code>FEATURES export in panels.js</code>	Usato solo dal componente orfano Features.jsx, rimosso insieme

Regola: se vedi un file `components/home/<X>.jsx` non elencato nella tabella INTERMEDIO sopra, NON modificarlo finché non hai chiesto a Trilli — potrebbe essere un orfano nuovo.

Caso 1 — Aggiornare il prezzo Test di Ferro

Vecchio: 27€ → Nuovo: 29€. File da modificare:

#	File	Cosa cambiare
1	<code>constants/faq.js</code>	Risposta domanda "Quanto costa?"
2	<code>components/home/Hero.jsx</code>	Card promo (se prezzo visualizzato)
3	<code>pages/MembershipPage.jsx</code>	Sezione downsell consulenza

Caso 2 — Aggiungere una nuova FAQ

Solo file `constants/faq.js`. Aprire, copiare l'ultimo blocco `{ q: ..., a: ... }`, duplicarlo, modificare `q + a`. Salvare.

```
{
  q: 'Posso pagare a rate?',
  a: 'Sì, tramite Klarna o Scalapay disponibili al checkout.',
},
```

Caso 3 — Rinominare un pannello

Es. "FERRO CORE" → "FERRO BASE". File:

#	File	Cosa cambiare
1	<code>constants/panels.js</code>	name: 'FERRO CORE' → 'FERRO BASE'
2	<code>constants/membership.js</code>	Cercare "FERRO CORE" nei includes
3	<code>constants/faq.js</code>	Cercare "FERRO CORE" nelle risposte
4	<code>components/home/FerroCore.jsx</code>	Cercare "FERRO CORE" nel JSX (potrebbe ripetersi)

Tip: in VS Code usare Cmd/Ctrl + Shift + F per cercare "FERRO CORE" in TUTTI i file del progetto contemporaneamente. Permette anche "Replace in Files".

Caso 4 — Cambiare CTA bottone

Es. "FAI IL TEST DI FERRO" → "INIZIA IL TUO TEST". Cmd/Ctrl + Shift + F sul progetto, cercare la stringa esatta, sostituire ovunque.

7. Errori comuni da evitare

■ Modificare i tag JSX

Toccare `<p>`, `<h2>`, `<div>`, ecc. rompe il rendering. Modificare SOLO il testo tra apertura e chiusura tag.

■ Cancellare virgolette o virgole

Nei file .js, ogni stringa ha virgolette di apertura e chiusura, e ogni voce di array termina con virgola. Cancellarle rompe la sintassi.

■ Apostrofi senza escape

Nei file .js con apici singoli `'...'`, gli apostrofi italiani vanno preceduti da backslash. Es: `'l'accesso'` non `'l'accesso'`. Alternativa: usare apici doppi `"l'accesso"` per non dover fare escape.

■ Push diretto su main

Non funziona se branch protection è attivo. Comunque: SEMPRE branch dedicato + PR.

■ Branch con nome generico

Evitare nomi come `edit`, `fix`, `test`. Usare `copy/<cosa-cambio>` sempre.

■ Modifiche in più file insieme senza descrivere

Se la PR cambia 5 file, il messaggio commit + descrizione PR devono spiegare cosa e perché.

■ Modificare la pagina Privacy senza OK Giovanni

Testo legale. Cambiarlo senza approvazione legale = rischio sanzione GDPR.

8. Checklist rapida (ogni modifica)

- 1. git pull origin main → partire allineati
- 2. git checkout -b copy/<descrizione> → nuovo branch dedicato
- 3. Modifica SOLO testo tra virgolette o tag JSX
- 4. Verifica nessun errore visivo nell'editor (linee rosse, ecc.)
- 5. git add <file> + git commit -m "copy: ..."
- 6. git push -u origin <branch-name>
- 7. Apri PR su GitHub con titolo descrittivo
- 8. Aspetta link preview Vercel (~1 min) e clicca per verificare
- 9. Tag Trilli per review (@trilligianni)
- 10. Trilli approva + merge
- 11. Verifica live su salutediferro.com (~3 min dopo merge)

Appendice A — Glossario

Termine	Significato
Branch	Una "copia parallela" del codice dove fare modifiche senza toccare il sito live.
Commit	Snapshot delle modifiche con un messaggio descrittivo. Più commit = cronologia.
Push	Caricare i commit locali sul repo GitHub remoto.
Pull	Scaricare le ultime modifiche dal repo GitHub al locale.
PR (Pull Request)	Richiesta di merge di un branch in main, con review.
Merge	Unire le modifiche di un branch in main.
Main	Il branch principale = il codice attualmente live su salutediferro.com.
Vercel preview	URL temporaneo che mostra come appare il sito CON le modifiche del branch.
Diff	Confronto visivo tra prima e dopo: rosso = rimosso, verde = aggiunto.
HMR (Hot Module Reload)	Quando si lavora in locale (npm run dev), salvare un file aggiorna il browser automaticamente.
JSX	Sintassi React: HTML mescolato a JavaScript. Es: <code><p>{var}</p></code> .
Constants	File di soli dati (testi, numeri, oggetti). Più sicuri da modificare.
Repo	Repository GitHub = il "contenitore" del codice del sito.

Appendice B — Comandi Git utili (cheat sheet)

Comando	Cosa fa
<code>git status</code>	Mostra cosa hai modificato e su che branch sei
<code>git pull origin main</code>	Aggiorna il repo locale con ultime modifiche da GitHub
<code>git checkout main</code>	Torna sul branch main
<code>git checkout -b copy/<nome></code>	Crea nuovo branch e ci si sposta sopra
<code>git checkout <branch></code>	Cambia branch (senza crearlo)
<code>git add <file></code>	Mette il file in coda per il prossimo commit
<code>git add .</code>	Mette TUTTI i file modificati in coda (usare con cautela)
<code>git commit -m "..."</code>	Crea uno snapshot con messaggio
<code>git push -u origin <branch></code>	Carica il branch su GitHub la prima volta
<code>git push</code>	Carica i nuovi commit (dopo il primo push)
<code>git log --oneline -10</code>	Vede gli ultimi 10 commit
<code>git diff</code>	Mostra cosa hai modificato ma non ancora committed
<code>git branch</code>	Elenca tutti i branch locali
<code>git branch -d <nome></code>	Cancella un branch locale (già mergiato)
<code>git stash</code>	Mette via temporaneamente le modifiche ("in tasca") senza committare
<code>git stash pop</code>	Riprende le modifiche messe via con stash

Appendice B.1 — In caso di errore

Problema	Soluzione
Ho modificato file sbagliati	<code>git checkout -- <file> # annulla modifiche non committate</code>
Ho committato per errore su main	Avvisare Trilli, non fare push
Conflitto al merge	Avvisare Trilli, non risolvere da soli
Vercel preview non si aggiorna	Aspettare 2 min, poi controllare commento Vercel sulla PR
Build Vercel fallisce	Aprire link "Details" sul check fallito sulla PR e leggere l'errore. Probabile syntax error JSX.

Per dubbi o blocchi: contattare Trilli. È sempre meglio chiedere che pushare un build rotto.